

GDN Tree-Scan: Correct State Commitment and Performance Limits of Tree Speculation in Recurrent Hybrids

Zhiyuan Ma

zhiyuanma0520@gmail.com

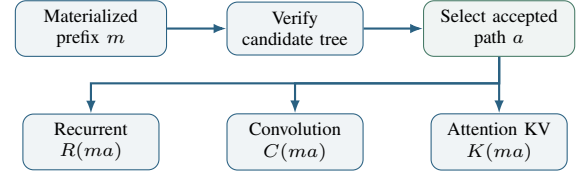
Abstract—Tree speculative decoding for recurrent-hybrid language models must preserve candidate ancestry and the state of the selected continuation. We study GDN Tree-Scan, a Gated DeltaNet verifier integrated with vLLM and FlashAttention-2. Its accepted-prefix contract coordinates recurrent state, convolution history, attention key–value storage, and a pending correction token. Archived regression witnesses expose incomplete state publication and numerical disagreement caused by physical attention layout. The publication repair changes a token fixture from 15 failures in 15 checks to zero; the layout repair yields shared-spine byte agreement in its tested route. An archived batch-four-configured Qwen3.6-27B-FP8 campaign on GB10 has a lower structural-token/pure-wall rate proxy for the tree than for native five-step prediction, 32.85 versus 42.74. Its numerator and denominator use different event populations, preventing a directly measured throughput interpretation. Fresh same-input measurements compare sequential scan/replay with local triangular-solve and finite-Neumann reconstructions on eight pilot prefixes and 31 of 32 confirmation prefixes passing frozen provenance checks. With fp32 products, errors against a float64 oracle are on the order of fp32 roundoff; tf32 increases error by roughly three orders of magnitude. Nevertheless, frozen bitwise, ordering, padding, and timing criteria fail. A retrospective audit also identifies omitted checks and inaccurate pilot rationale in the frozen record. Historical sampling and binary provenance limit the archived comparison; current model-level continuation and matched final-stack timing remain unqualified. The results provide scoped state-integrity witnesses and numerical/cost characterization, without a superiority or distribution-equivalence claim.

Index Terms—speculative decoding, Gated DeltaNet, state commitment, numerical reproducibility, language-model serving

I. INTRODUCTION

Speculative decoding reduces serial target-model invocations by verifying draft tokens in parallel. Its sampling guarantee assumes that verification supplies the intended target probabilities. Tree speculation expands the candidate set, but it does not relax that assumption [1]–[3]. For recurrent-hybrid models, realizing the verifier requires more than an attention mask: a candidate must inherit the state of its ancestors, and the next serving iteration must inherit exactly the path selected for continuation [4], [5].

This paper studies that boundary in GDN Tree-Scan, a Gated DeltaNet verifier integrated with vLLM and a fork of FlashAttention-2. The implementation uses branch-local



Same selected materialized prefix on all three surfaces; correction token b may remain pending for the next forward.

Fig. 1. A selected token path must have a matching continuation state. Recurrent, convolution, and attention-KV state must all describe the same materialized prefix. A newly sampled token can remain pending until the next forward, just as in native decoding.

recurrent computation, device-side multidraft selection, and accepted-path publication across recurrent state, convolution history, and attention key–value (KV) storage [6], [7]. An attention-KV mapping error demonstrates why repairing only the recurrent state is insufficient: a valid selected token path can still read a rejected sibling on the next iteration. A second failure arises even when masked branches contribute zero attention weight. Moving the visible spine keys changes floating-point reduction association and can alter a near-tied verification decision [8].

Acceptance length also leaves the performance question incomplete. In one archived campaign configured for batch four, accepted drafts plus one per event averaged 5.286 for the tree and 4.422 for native five-step multi-token prediction. The archived rate proxy was lower for the tree: 32.85 versus 42.74 structural tokens/s [9], [10]. This proxy divides the global accepted-plus-one mean by a mean cost drawn from a different, pure-decode event population; it is not a directly measured emitted-token throughput. The support mismatch motivates matched accounting for output, wall time, and component costs in current-system measurements.

The revision makes four bounded contributions. First, it states the accepted-prefix contract at the actual serving boundary and associates it with concrete regression witnesses. Second, it explains a physical-layout source of numerical spine disagreement and the tested repair. Third, it reconciles archived acceptance and timing results, including a strong depth-matched chain control and a later paired attention-kernel improvement. The latter reduced its tested batch-four step time by 27.03 ms, but

does not establish tree-over-chain superiority [11]. Fourth, a fresh same-input kernel study characterizes scan, triangular-solve, and finite-Neumann arithmetic against distinct native references, including failed frozen criteria and an audit of omitted checks.

Parallel GDN tree verification and compact accepted-state reconstruction are already addressed by direct related work. Section II positions this study accordingly. We do not claim a first hybrid-tree system, general bitwise equivalence, or a measured advantage over those implementations. Section VII separates the experiments needed for stronger current-system claims from the historical findings already available.

II. BACKGROUND AND RELATED WORK

A. Drafting, verification, and serving

Speculative sampling constructs an exact target sample from proposal and residual distributions, conditional on the probabilities actually supplied [1], [2]. SpecInfer supplies the tree-verification framing; Sequoia, EAGLE-2, and OPT-Tree investigate tree construction and its performance consequences [3], [12]–[14]. Our study keeps these algorithmic questions separate from whether a deployed kernel and cache return the intended probabilities.

PagedAttention and SGLang address the scheduling, memory, and execution substrate [15], [16]. Hybrid serving additionally manages recurrent-state pools alongside KV storage, as documented by SGLang’s hybrid-model integration [17], [18]. Thus neither a fast isolated recurrent kernel nor low transient state memory alone determines user-visible latency. FlashAttention-2’s work partitioning supplies the attention substrate examined here [19].

B. Recurrent and hybrid speculation

Gated DeltaNet augments a decayed recurrent state with a data-dependent delta update [4]. Parallel delta-rule formulations provide relevant algebraic background [20]. Stateful speculation is established territory: STree studies speculative trees for hybrid state-space models, SpecMamba targets an FPGA implementation, and component-aware self-speculation studies how hybrid components affect the design [5], [21], [22]. These works motivate an explicit state lineage; they do not justify treating every recurrent update or numerical implementation as interchangeable.

C. Direct GDN comparisons

Bole derives a tree-structured closed form and evaluates it with a value-tiled finite-polynomial solver. It stores compact update factors and reconstructs the accepted state. Its SGLang integration combines batch-wide verification budgeting with captured GPU execution. Evaluation includes unquantized Qwen3.5-27B on GB10 and online replay of agent sessions [23]. Its state and serving mechanisms directly overlap the problem studied here. Our measured replay cost is therefore not a lower bound on alternative implementations.

Concretely, Bole’s correction system is $(I + G)U = R$, where G contains strict-ancestor interactions and R is the

TABLE I
DIRECT COMPARISON SCOPE. EXTERNAL METHODS WERE READ, NOT EXECUTED IN THIS REVISION.

Method	Overlapping mechanism	Boundary for this study
Bole [23]	Parallel verifier; factorized commit	Different stack and precision; no matched run
TreeWY [24]	Tree solve; accepted-state reconstruction	Different models and operating regime; no matched run
ReplaySSM [25]	Input caching; deferred state work	Alternative state policy; no matched run
GDN Tree-Scan	Scan/replay; accepted-path remapping	Specific numerical and lifecycle witnesses; historical cost evidence

source correction. Tree depth d gives $G^{d+1} = 0$, so $U = \sum_{m=0}^d (-G)^m R$. This finite identity changes the arithmetic schedule from sequential updates to matrix propagation [23]. Our path preserves sequential updates and replays accepted operands; Section IV-D explains why the numerical comparison remains an empirical question.

TreeWY expresses GDN tree verification as a triangular system and reconstructs the accepted state from pseudo-values. Its vLLM evaluation uses Qwen3.5-35B and 397B on B200, greedy verification, and disabled prefix caching. It explicitly permits finite-precision disagreement with the baseline and reports that wider trees improve acceptance without yet winning throughput [24]. Consequently, shared-spine byte checks in our particular route neither establish that TreeWY is incorrect nor replace its serving comparison.

ReplaySSM caches recurrent inputs and changes when state is materialized; its implementation RFC includes GDN and Mamba2 [25]. Reconstruction and replay are not claimed as new concepts here. Table I states the comparison boundary. Published speed ratios from different checkpoints, precision formats, workloads, and software builds are not combined into a performance ranking.

III. THE ACCEPTED-PREFIX CONTRACT

A. State and the execution boundary

Let m be a materialized token prefix: the tokens already consumed by the target’s forward computation. Write its persistent state as

$$S(m) = (R(m), C(m), K(m)), \quad (1)$$

where R collects recurrent matrices, C collects convolution histories, and K collects attention KV entries. The logical emitted prefix can also include a newly sampled token b that has not yet been consumed. The continuation boundary is then $(S(m), b)$, not a claim that $S(mb)$ has already been computed. This distinction is essential when counting the correction or bonus token while checking cache state.

A tree descriptor contains candidate token IDs, parent indices, depth/position information, attention visibility, and maps between logical nodes and physical slots. For a candidate node i , each target layer must consume the state obtained

from the candidate’s root-to-parent path. At publication, the selected materialized path, pending-token convention, and next-drafter input row must agree. Figure 1 illustrates the three state surfaces documented by the implementation audit [7], [10].

B. The recurrence and path locality

For one GDN head, let $R \in \mathbb{R}^{d_v \times d_k}$. Queries and keys are in $\mathbb{R}^{d_k}$ and values in $\mathbb{R}^{d_v}$. With decay α_i and write strength β_i , a node applies the standard update [4]

$$R'_i = \alpha_i R_{\pi(i)}, \quad \delta_i = \beta_i (v_i - R'_i k_i), \quad (2)$$

$$R_i = R'_i + \delta_i k_i^\top, \quad o_i = R_i q_i, \quad (3)$$

where $\pi(i)$ is its parent. A previous node in the packed buffer need not be $\pi(i)$. The causal convolution likewise requires path predecessors rather than adjacent packed rows. The attention visibility relation must select the same logical prefix and ancestry. These requirements follow from the model’s computation; a correct attention mask cannot repair a wrong recurrent parent.

The implementation verifies nodes with branch-local scratch and makes the selected continuation durable through replay and remapping. This is an implementation route, not the only way to satisfy the contract [6], [7]. In particular, rejecting a branch requires that its contents cannot influence future computation; it does not require zeroing every scratch byte if future reads are correctly restricted.

C. Four different correctness questions

Algebraic correctness asks whether the recurrence or its reformulation computes the same real-arithmetic function. *Numerical agreement* asks whether particular kernels, layouts, and precisions agree with a selected reference. *Sampler correctness* asks whether the acceptance/residual rule returns its supplied target distribution. *Serving correctness* asks whether these properties survive cache restoration, row reuse, batching, and the next iteration [2], [10], [20].

For example, for target probabilities p and proposal probabilities q , accepted mass is $\min(p, q)$ under ordinary rejection sampling. Sampling the remaining mass proportional to $(p - q)_+$ gives the target marginal. The argument is conditional on the actual proposal law and target probabilities; substituting stale logits or an incorrect state changes the distribution being sampled [1], [2]. The same distinction applies to the multidraft reference used here [3], [6].

If every iteration supplies the reference conditional probabilities and publishes an equivalent continuation boundary, an induction over emitted tokens gives the reference joint distribution. This is a conditional argument, not a proof that the implemented system satisfies its premises. A tolerance test, zero observed garble, or equal task outcomes does not discharge all those premises.

IV. IMPLEMENTATION AND FAILURE MECHANISMS

A. A descriptor shared by verification and commitment

Figure 2 shows the served path. Native multi-token prediction (MTP) supplies the principal draft chain; selected variants

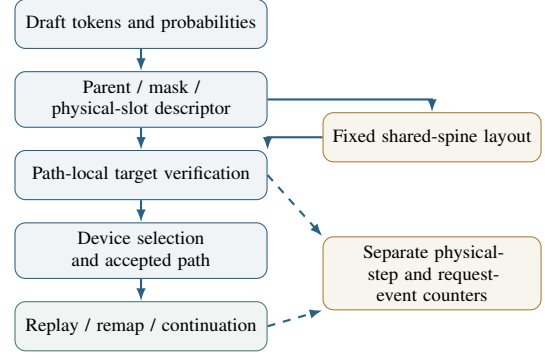


Fig. 2. One descriptor connects the target forward and the continuation state. The selected row also controls the next drafter input. Timing records must identify whether they count a physical step or a request verification event.

Algorithm 1 Logical verifier and continuation boundary

Require: Reference-aligned state boundary, candidate descriptor T

- 1: Map logical candidates to physical slots and positions.
- 2: Verify using path-local convolution, GDN, and attention.
- 3: Apply the configured sampler to matching target/proposal rows.
- 4: Obtain the accepted path and any pending correction token.
- 5: Publish recurrent and convolution state for the materialized path.
- 6: Map its attention KV entries to the native continuation layout.
- 7: Map the selected hidden-state row to the next drafter input.
- 8: Preserve the native pending-token and position convention.
- 9: **return** Continuation boundary and emitted-token accounting

add runner-up candidates. The descriptor maps each candidate through the attention mask, recurrent scan, sampler, and selected-path publication. Historical variants also use a heuristic suffix predictor; this changes the proposal source and must be recorded separately from tree topology [6], [9].

The attention fork applies an additive ancestry bias to speculative suffix scores while leaving the context visible. It retains the native prefill route. Recurrent nodes obtain their parent state from the pre-tree state or node-local scratch. After device selection, accepted-path replay and remapping establish the continuation state [6]. Cache checkpoints and restored rows must obey the same contract; a correct cold first request does not establish correctness after a checkpoint hit or page reuse [7].

B. Accepted state with the wrong attention KV

The archived remap failure left recurrent and convolution state aligned with the selected path, while attention read a neighboring sibling’s KV entry after a non-contiguous path was accepted [7]. The fix copies each selected node’s KV entry

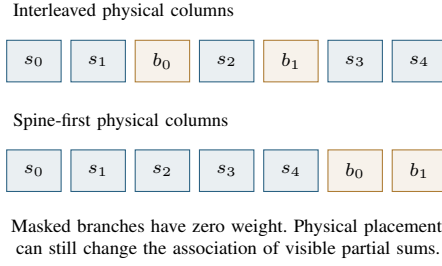


Fig. 3. Illustrative physical slot layouts for one spine query. Blue entries are visible spine keys and amber entries are masked branches. The repair keeps shared-spine positions fixed within the tested execution layout. The drawing is a mechanism illustration, not a new numerical measurement.

from its verification slot into its linear continuation slot. Its role is state movement rather than a new attention equation. Algorithm 1 makes the missing step explicit.

The associated token fixture changed from 15/15 failures to 0/15 across the reported eager, graph, and cold-cache checks. In served-script checks, the repaired cat8 and cat6 arms had zero observed corrupted-identifier flags among 84 and 61 scripts respectively [7]. The pre-fix observations came from earlier runs, so their rates should not be treated as a randomized estimate of the repair’s effect. The fixture supplies the more direct causal witness.

A later launcher audit found that previously validated remap and reorder flags were enabled in narrow launchers but disabled in the general campaign launcher. The corrected campaign therefore records post-boot engagement, not merely source-code availability [10]. This is why the identity of a compiled implementation and the effective configuration belong to the evidence boundary.

C. Masked branches can change the shared spine

In exact arithmetic, adding masked keys contributes zero to attention. In the tested FlashAttention-2 route, however, physical score columns also determine the association of the online-softmax reduction [8], [19]. Interleaving extra branch slots changes the positions of surviving spine keys. The visible values are unchanged, but their partial sums need not be associated identically.

The recorded witness shows a one-unit-in-the-last-place discrepancy in a bfloat16 output at magnitude eight: $2^{3-7} = 0.0625$. The diagnostic identifies the shared attention spine and reports a greedy acceptance difference of 0.087 drafts/event between the compared tree variants [8]. Independent stochastic task traces cannot establish the same claim. A meaningful superset check fixes the common prefix, proposals, target reference, and decision rule. Under greedy longest-matching-path selection with unchanged shared-node outputs, adding candidates cannot remove an existing matching path. A numerical change can violate that premise.

The repair places the shared spine contiguously before other tree slots and propagates the permutation through the relevant maps. In the recorded observation harness and in-process check, the reordered shared spine matched the spine-only

reference byte-for-byte; an interleaved control retained the discrepancy [8]. This is evidence for the tested route and shape family. Different compiler choices, kernel buckets, quantization formats, or sampling operations require their own checks. A fixed layout alone is not a universal floating-point identity theorem.

D. Reconstruction, rounding, and accumulated disagreement

The archive includes an earlier local WY prototype. Its investigation reports a recurrent-state error of approximately 1.66×10^{-3} and a subsequent maximum logit discrepancy of 3.32. The diagnosed state-write path reused an excessively rounded output representation. A separate state calculation reduced the isolated error; later assessments retained the algebraic equivalence and left broader acceptance-based validation open [10]. These are documentary observations about our historical prototype, not a reproduction or refutation of TreeWY. The evidence manifest identifies both the failure record and its later correction.

Other archived layer-localization studies examined our sequential tree route as well. They motivate a distinction between a local numerical discrepancy and its effect after further model layers or recurrent steps. A first-order explanatory model is $e_{\ell+1} \approx J_{\ell} e_{\ell} + \delta_{\ell}$: later layers can amplify an existing residual while introducing additional error. This expression is not a fitted error bound. Small local errors, or matching current outputs, do not establish matching future logits [10].

The numerical reference must also be named. The pinned native speculative-update and one-token decode paths differ in gate rounding and state-storage boundaries. Our scan and replay share a sequential update body, but their compiled agreement still requires GPU checks [10]. Thus a real-arithmetic reconstruction identity does not by itself establish deployment agreement; conversely, disagreement with our selected reference does not establish an algebraic defect. E7a and E7b isolate verification arithmetic, state commitment, and subsequent continuation before measuring any deployment benefit. The E7a kernel-level pilot (Section VI-F) measures the first two of these boundaries: the two pinned native paths differ from each other by 2.2×10^{-4} to 1.7×10^{-3} of the state maximum on the tested operands, and an isolated ablation shows the difference is the decode kernel’s documented bfloat16 rounding of the write gate (an oracle applying only that rounding matches it at the fp32 floor).

V. EVALUATION PROTOCOL AND EVIDENCE STATUS

A. Campaigns are separate experimental objects

The main empirical material concerns Qwen3.6-27B-FP8 on GB10/DGX Spark [10], [26]. Table II distinguishes the archived campaigns. They use different implementation stages and, in some cases, different harnesses. They are not replicates of a single final system. The accompanying evidence manifest records file paths and SHA-256 hashes; the accounting script recomputes the H3 table from stored JSON without running inference.

TABLE II

EVIDENCE CARRIED INTO THIS DRAFT. “VERIFIED ARCHIVE” MEANS THE REPORTED OBSERVATION WAS CHECKED AGAINST AN AVAILABLE SOURCE; IT DOES NOT MEAN THE EXPERIMENT WAS RERUN ON THE CURRENT CODE. P0 AND THE KERNEL-LEVEL E7A STUDY ARE EXECUTED; E2/E7B, E1, AND CONDITIONAL E3 REMAIN PLANNED; E7C IS A DEFERRED EXTERNAL COMPARISON.

ID	Evidence	Supported statement	Important boundary
H1	KV-remap report [7]	Fixture repair and observed post-fix script checks	Not a distribution proof or a randomized quality comparison
H2	Spine-layout report [8]	Numerical witness, shared-spine repair, later acceptance values	Byte agreement scoped to tested reference and route
H3	Postfix3 JSON and close-out [10]	Three-arm acceptance and mixed-population rate proxy	Historical nominal-B4 campaign; unmatched event populations and duplicated constraints
H4	Paired attention optimization [11]	27.03 ms local step reduction in its configuration	Different stage from H3; not a composed tree-over-chain result
H5	Repeated-task audit [10]	Limits of the 16-task quality sample	Same Astropy subset; heterogeneous historical arms
H6	Archived numerical investigations	Failure mechanism and later correction	Documentary evidence; original captures not reproduced
P0	Configuration audit (Section V)	Sampling reconstruction; no new model measurements	Historical binary gaps retained
E7a	Same-input kernel study (Section VI-F)	Historical/synthetic controls, eight fresh pilot prefixes, and 31 of 32 confirmation prefixes passing the frozen provenance checks	Local realizations, not the authors’ systems; frozen numerical/control criteria missed; no model-level qualification
E2/E7b; E1; E3	Planned	—	Not executed

The H3 campaign configures four concurrent requests, enables prefix caching, and runs a 16-task Astropy subset drawn from SWE-bench Verified through qwen-code. Native MTP-5 uses five draft steps. MTP-11 matches the maximum draft depth of the tail6 tree, whose later suffix uses a different proposal source [9], [10]. It is a useful strong control, but not a pure topology ablation. Task resolution follows the benchmark evaluation; the small chosen subset does not represent the full benchmark [27].

B. Sampling provenance remains an explicit limitation

Historical reports label several campaigns with requested temperature 0.6. The archive documents duplicated sampling constraints [28]. P0 reconstructs the three July patchers at their recorded, distinct source revisions against the launchers’ pinned image: each produces two target-logit constraint calls, while the selected current revision produces one. This is source reconstruction, not recovery of the original patched runtime. The target-row temperature divisors compound to 0.36, but top-k/top-p filtering also repeats and bonus paths differ; the run is not equivalent to uniform temperature-0.36 sampling. We retain the requested settings as *nominal*. Historical loaded-extension and model-file identities remain unresolved; current file hashes cannot establish July identity. The artifact includes the source reconstructions and per-arm audit manifests.

The stored numbers remain observations of the archived runs, useful for describing behavior and diagnosing cost. They do not provide a controlled-temperature estimate of the final system’s advantage. Likewise, an available correctness test counts as a current passing result only when its execution record establishes the relevant route [10].

C. Tokens, events, physical steps, and service time

Let a_e be accepted drafts and c_e be actual emitted tokens in request verification event e . For ordinary nonterminal events, $c_e = a_e + 1$ because of a correction/bonus token. End-of-sequence and truncation instead require the actual count. A physical GPU step can contain several request events. A measured decode rate requires output counts and wall intervals from the same event population; component attribution also requires matching the timed operations to those events.

H3’s archived instrument does not supply that matching. With A_{all} the global accepted-draft count, D_{all} the global draft-event count, W_{pure} retained pure-decode wall seconds, and D_{wall} their event count, its reported quantity is the accounting proxy

$$R_{\text{H3 proxy}} = \frac{A_{\text{all}}/D_{\text{all}} + 1}{W_{\text{pure}}/D_{\text{wall}}}. \quad (4)$$

The wall instrument retains start-to-start intervals between consecutive pure-decode steps, caps idle intervals, and breaks the chain on mixed or prefill steps. Retained wall events cover only 74.19%, 60.80%, and 63.30% of global draft events for MTP-5, MTP-11, and the tree. Accepted-token totals for that retained subset are absent. Moreover, accepted tokens plus draft events differ from recorded generation-token counts by 978, 1923, and 2115 tokens, respectively; the aggregate does not identify the complete cause. Replacing the structural numerator with global generation tokens would still leave unmatched populations [10].

An earlier analysis also mixed milliseconds per physical step and per request event. Dividing draft/commit span means by measured pure-forward events/step repairs the units, but does not align their populations: these spans include mixed steps, whereas the verification timer includes pure-decode forward steps. The plotted residual is wall cost minus those

TABLE III

ARCHIVED CORRECTNESS AND ACCEPTANCE OBSERVATIONS. COUNTS HAVE DIFFERENT UNITS; THEY ARE NOT POOLED INTO ONE FAILURE RATE.

Observation	Recorded result
KV-remap token fixture	15/15 failures to 0/15
Post-fix cat8 script flags	0/84 corrupted identifiers
Post-fix cat6 script flags	0/61 corrupted identifiers
Reordered shared spine	Byte match in tested checks
Later B4 cat8 acceptance	3.4996 drafts/event
Later B4 cat6 acceptance	3.3332 drafts/event

normalized component estimates, not measured host cost. The resulting GPU-estimate/wall-proxy ratios of 1.017–1.025 do not validate the representativeness assumption. The earlier approximately 0.53 unit mismatch is superseded, while the population mismatch remains unresolved. A retrospective addendum preserves the raw numbers and documents the counters and historical instrument sources in the artifact. TTFT, TPOT, queueing, cache misses, and full-service throughput remain separate quantities.

VI. RESULTS

A. State and layout findings

Table III summarizes H1 and H2. The repaired state mapping eliminates the fixture’s observed failure, while the layout investigation identifies a separate numerical cause. These results address distinct failure modes. A system can satisfy the recurrent-path equation yet violate attention-KV publication, and can satisfy an abstract mask yet change the finite-precision output [7], [8].

After the layout repair, the recorded B4 cat8–cat6 acceptance difference is 0.1664 drafts/event. This is consistent with the restored opportunity of the wider tree, but it is not a paired estimate of the greedy witness’s effect: the deployed arms follow different stochastic trajectories [8]. Equal task counts in that campaign likewise cannot establish equivalence. A direct structural claim rests on the matched-input numerical check.

B. The archived tree has the lowest rate proxy

Table IV reports H3. The tree’s global accepted-plus-one mean is about 19.5% higher than native MTP-5’s, while its retained pure-wall cost per event is about 55.5% higher. Their ratio yields a lower proxy. MTP-11 has the largest structural numerator, but its proxy is also lower than MTP-5’s [9], [10]. These observations retain the archived ordering; the unmatched populations prevent interpreting it as a directly measured throughput ranking.

Figure 4 gives normalized component estimates. Verification averages 106.28 ms/event for the tree and 66.68 ms/event for MTP-5; the commitment estimates are 16.89 versus 2.49 ms/event, and drafting 35.04 versus 31.72 ms/event. MTP-11’s draft estimate is 68.26 ms/event [10]. These describe the sampled operations, but their different supports do not establish a causal decomposition of the wall-time difference or a lower bound for another representation.

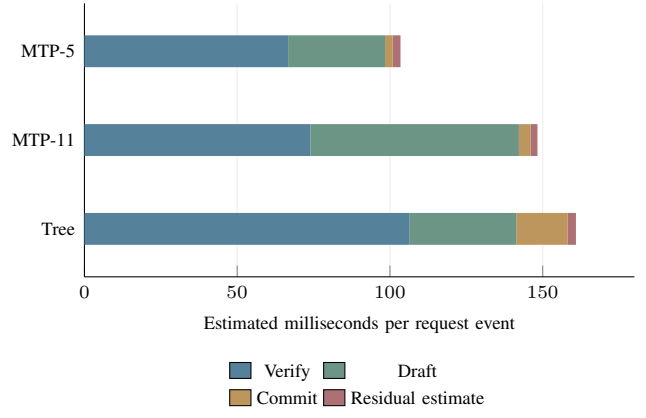


Fig. 4. H3 component estimates converted to milliseconds per event. Draft/commit spans, verification events, and wall intervals have different supports. The residual closes their sum by definition; it is not measured host time.

Configured batch size is not realized occupancy. The measured request events per pure-decode step range from 2.74 to 2.80 in this campaign. Larger nominal capacity cannot be substituted for the observed denominator. The recorded task totals are also descriptive: one run per arm on the same selected subset does not support a significance claim or a quality ranking [9], [10].

C. A conditional break-even calculation

Define $\bar{a} = A_{\text{all}}/D_{\text{all}}$ and $\bar{t} = W_{\text{pure}}/D_{\text{wall}}$ for each arm. At fixed values of these different population means, Eq. (4) gives a conditional structural acceptance threshold

$$\bar{a}_{\text{tree}}^* = (\bar{a}_{\text{native}} + 1) \frac{\bar{t}_{\text{tree}}}{\bar{t}_{\text{native}}} - 1. \quad (5)$$

Using the unrounded H3 values yields 5.88 accepted drafts/event for proxy parity with MTP-5. Alternatively, at the observed global tree acceptance, its retained pure-wall mean would need to fall to 123.66 ms, a 23.1% reduction. The implementation of this calculation in the paper artifact corrects a historical note that omitted the bonus token [10].

This is a conditional threshold for the proxy, not a validated throughput break-even point. Increasing depth, changing the drafter, changing batch occupancy, or reducing cache capacity can change both numerator and denominator. It is not a forecast that the tree can reach the threshold at unchanged cost, nor a claim that no alternative tree algorithm can do better.

D. A reliable local optimization, with composition still open

H4 examines attention work partitioning at a later implementation stage. Pairing query-head work uses previously idle query-tile capacity and reduces the number of thread blocks in the tested B4 route [11]. Its paired timing summary reports a mean reduction of 27.03 ms/step across four passes, with standard deviation 13.77 ms and a reported one-sided 95% lower bound of 10.82 ms. The summarized baseline and optimized times are 408.313 and 381.284 ms. These are physical-step values for that campaign; subtracting 27.03 ms

TABLE IV

H3: HISTORICAL NOMINAL-B4 CAMPAIGN, PREFIX CACHE ENABLED, 16 ASTROPY TASKS. VALUES ARE RECOMPUTED FROM THE CORRECTED RAW JSON. NOMINAL TEMPERATURE WAS 0.6; P0 RECONSTRUCTED DUPLICATED TARGET-CONSTRAINT PASSES, WHILE HISTORICAL LOADED-BINARY IDENTITY REMAINS UNKNOWN. THE RATE PROXY IS EQ. (4): THE GLOBAL ACCEPTED-PLUS-ONE MEAN DIVIDED BY MEAN COST ON A DIFFERENT PURE-WALL SUBSET. OCCUPANCY USES A THIRD, PURE-FORWARD POPULATION.

Arm	Accepted/event	Accepted + 1	Pure-wall ms/event	Rate proxy (s^{-1})	Events/step	Resolved
Native MTP-5	3.422	4.422	103.45	42.74	2.79	10/16
Native MTP-11	4.923	5.923	148.28	39.95	2.74	8/16
Tree, tail6	4.286	5.286	160.88	32.85	2.80	10/16

TABLE V

LATER OPTIMIZATION EVIDENCE. PROVISIONAL AND UNMEASURED ITEMS ARE EXCLUDED FROM FACTUAL COMBINED-SPEED CLAIMS.

Change	Evidence status
B4 query-head pairing	Verified archive: 27.03 ms paired mean reduction
B4 single-pass GDN scan	Provisional 8.984 ms observation; final paired validation pending
Batched tree-accept walk	Correctness checks archived; deployment timing pending
Combined final stack vs. chains	Planned E1/E3; no measured conclusion

from H3’s event-normalized time would mix experiments and units.

The later archive also contains a provisional single-pass GDN result and correctness work on a batched tree-accept walk, but not the completed composed comparison [10], [11]. Table V deliberately preserves those statuses. Kernel interactions, changed bottlenecks, and workload variation make addition of individual best deltas unjustified.

E. Quality and the original B1 observation

The original manuscript reported 23.88 versus 18.80 token-weighted request-decode tokens/s for a small June B1 gate, with 18.51 versus 17.80 under a per-request-equal view [6]. Those are different metrics and a different campaign from H3. We retain them as historical context, subject to the sampling audit, rather than as the current revision’s acceleration headline. Its recurrent-oracle rescore sample was not a distribution-distance proof.

H5 summarizes 21 historical arms on the same 16 Astropy tasks: seven tasks always resolved, four never resolved, and five varied [10]. Implementations differ across arms, so these cannot be pooled as 21 independent replications of one system. The sample is useful for detecting gross regressions; it is weak evidence for non-inferiority or general task quality. An optional newer quantization campaign also records external retrieval of a benchmark gold patch, requiring provenance review before its affected outcomes can be used. No such outcome supports the core conclusions here.

F. E7a: arithmetic on historical, synthetic, and fresh operands

After P0 we executed the E7a comparison at the kernel level on 21 September 2026 (artifacts of record, with hashed sources, compiled-kernel hashes, integer-view equality metrics, and

telemetry, under `experiments/out-20260921T230026Z-e7a-ladder-v3/`; the earlier same-operand run `out-20260921T222616Z-e7a-ladder/` is retained). Inputs are the rounded operands of one GDN layer step: bfloat16 q, k, v , bfloat16 raw gates, fp32 $A_{\log}$, bfloat16 gate bias, the fp32 pre-tree state, and the tree descriptor. Four operand sets are historical June-2026 captures at layers 62, 1, 12, and 0 of the served ten-node caterpillar (depth 5, B1). They are dependent operand sets whose independent-prefix count cannot be recovered from the files; they are not fresh prefixes and contribute nothing to the planned eight- and 32-prefix sets. Five further sets are labeled synthetic with historical-like gate statistics: chains of depth 1, 5, and 11, a binary tree of depth 3, and the caterpillar; a synthetic four-request batch exercises the batched kernels. Three references are kept apart: a float64 serial recurrence on the same rounded operands, the pinned native speculative-update kernel run per root-to-node chain, and the pinned one-token decode kernel run token by token.

A fresh eight-prefix pilot followed on 22 September 2026 on the intended configuration: the audited Qwen3.6-27B FP8 checkpoint (read-only), the pinned vLLM image, B1 eager execution with stock `TREE_ATTN`, and the served ten-node caterpillar on the stateless production tree route (distinct from the archived forked FlashAttention-2 route), and a documented one-hunk repair of the source patcher that made that route runnable at HEAD (a drafter constant emitted only under a fixed-tree mode; recorded with original and repaired hashes). Ninety-six candidate prefixes were cut from local repositories in three context strata and scored once by a native MTP reference server; eight pilot and 32 disjoint confirmation prefixes were frozen from native-reference behavior alone (top-1/top-2 margin and context strata) before any tree output was read. Each pilot prefix received one boot and one greedy request, and one one-shot capture of layer 62’s first tree step after the full prefill (two prefill forwards for the 4096-token prompts under the server’s logged 2048-token budget). A fail-closed provenance inspector (51 checks per capture: hashed prefix and prompt-token binding, payload written inside the first tree step, the per-request scheduler trace carrying the response identifier, per-step alignment of the accepted draft paths with the API-returned token identifiers, exact decoding of those identifiers to the response text, exact geometry and dtypes, non-trivial pre-state) passed on all eight (four prompts of 256 tokens, four of 4096), with three earlier boots retained as failed captures. The eight sets are independent frozen prefixes; the harness run

of record is `experiments/out-20260922T013313Z-e7a-fresh-pilot-v2/`.

Mechanism A is this worktree’s production scan and accepted-path replay. Mechanisms B and C are our Triton reimplementations of the two published families on the same masked system $(I + G)U = R$: forward substitution (the WY/TreeWY family) and the finite Neumann series $\sum_{m=0}^d (-G)^m R$ (d correction powers plus the identity, hence $d + 1$ terms; the Bole family), each followed by the compact reconstruction $S_a = P_a S_0 + \sum_{j \leq a} (P_a / P_j) u_j k_j^\top$. Dense products run either in fp32 or in tf32 tensor-core precision; the corrected June-2026 WY kernel is included as a historical realization. Every accepted prefix, including zero acceptance, is committed, first from each method’s own factors and then from identical oracle factors.

Table VI summarizes. In float64 the three mechanisms and the oracle agree to about 10^{-15} . With fp32 products every realization sits at the fp32 floor against the oracle on all nine sets: verifier outputs within $0.9\text{--}5.2 \times 10^{-7}$ of the reference maximum, correction factors within $1.7\text{--}3.4 \times 10^{-7}$, committed states within $1.0\text{--}3.7 \times 10^{-7}$. Commit from identical oracle factors is at the same scale, consistent with fp32 roundoff on these operands; this is not a general error bound. In a diagnostic comparison with fp32 output stores (a separately compiled specialization of both kernels, not the deployed bfloat16 path), the production scan’s outputs are bit-identical to the native speculative-update kernel’s on every element of all nine sets (integer-view equality); the deployed bfloat16-store outputs are bit-identical in 99.998–100% of elements (two sets each differ in a single element by one unit in the last place), and the bfloat16 outputs of B and C are bit-identical to native’s in 99.97–99.99% of elements. With tf32 products the same kernels lose three orders of magnitude (outputs $6.5 \times 10^{-4}\text{--}1.4 \times 10^{-3}$; bfloat16 agreement 74–79%). In these local realizations, dense-product precision materially changes agreement with the selected native reference. This ablation does not identify the authors’ implementation choices or establish model-level agreement.

The two pinned native paths disagree with each other by $2.2 \times 10^{-4}\text{--}1.7 \times 10^{-3}$ of the state maximum, with 75–94% bfloat16-output agreement. An isolated oracle-convention ablation attributes the gap: a float64 oracle that applies only the decode kernel’s documented bfloat16 rounding of the write gate matches it at $1.2\text{--}4.0 \times 10^{-7}$, whereas an oracle that applies only its $x/\sqrt{\cdot}$ normalization leaves the full gap ($2.2 \times 10^{-4}\text{--}1.7 \times 10^{-3}$) in place; in float64 the normalization convention moves the state by at most 5×10^{-15} (this identifies which declared convention the fp32 kernel follows; it does not measure the two fp32 kernels’ entire implementation difference). On these operands the gap is therefore the declared gate rounding, not an implementation error, and for the outputs where the two paths differ no single result can lie within the fp32 floor of both. Controls: sibling reordering is bitwise-invariant for A on all sets and for B and C on eight of nine (last-bit differences on the bushy binary tree from changed accumulation order); changing the padded node count changes last bits

($\leq 1.2 \times 10^{-7}$ absolute); dropping one Neumann term fails (factor error up to 0.29), so the nilpotency term is load-bearing; for all four requests of a synthetic batch, the batched outputs, factors, and commits are bit-identical (integer view) to the single-request results. Along the depth-11 chain the committed-state error grows from 5×10^{-7} to $1.0\text{--}1.3 \times 10^{-6}$ absolute, still about 10^{-7} relative. A separate synthetic tiny-gate pass (`out-20260921T231030Z-e7a-tinygates-v3/`; chains of depth 11 and 5 and the caterpillar with cumulative log-decay down to -912 , so that the path decay P_i underflows to exactly zero in fp32) produced no non-finite value in any realization: outputs and committed states stayed within $0.9\text{--}5.8 \times 10^{-7}$ of the oracle because every implementation evaluates path-decay ratios as $\exp(\log P_i - \log P_j)$ with the exponent masked before exponentiation, never as a literal quotient of underflowed products. The same run shows that with vanishing gates the strict-ancestor coupling vanishes too, so even the truncated Neumann series is accurate there; the truncation control fails only when coupling is present.

On the eight fresh sets the picture is unchanged and now bound to the live server. The production scan’s fp32-store output is bit-identical (integer view) to the native speculative-update kernel’s on every element of all eight sets, and so is its deployed bfloat16 store; the bfloat16 output the server actually emitted at that step (the payload’s served output) is byte-identical to both. Against the float64 oracle the production scan sits at $1.1\text{--}2.2 \times 10^{-7}$ of the output maximum, B and C with fp32 products at $2.2\text{--}3.6 \times 10^{-7}$, their compact commits at $1.1\text{--}1.7 \times 10^{-7}$, and the replay commit at $1.5\text{--}2.9 \times 10^{-7}$; the bfloat16 outputs of B and C have a minimum bitwise agreement with native of approximately 99.9788%; tf32 products again lose three orders ($6.7\text{--}8.6 \times 10^{-4}$; bfloat16 agreement 77–81%). The two native paths disagree by $1.3 \times 10^{-4}\text{--}1.3 \times 10^{-3}$ of the state maximum and the write-gate rounding again explains it ($1.3\text{--}2.9 \times 10^{-7}$ against the seam oracle). Sibling reordering is bitwise-invariant on all eight sets and every replay repeats bitwise. Two limits are specific to the fresh operands: the stateless production route materializes no per-node tree state, so a served-state comparison is not available (only the served output is); and two boots of the same 256-token prefix at temperature zero diverged at the sixth emitted token, after the captured step, with different diagnostics active in each boot, which we record without attributing (cross-boot greedy divergence on this stack is a known limit and belongs to E2/E7b). Before reading any confirmation output we froze, from these eight sets, the candidates (A, B, and C with fp32 products), per-class tolerances at the pilot maximum times a declared margin (zero units in the last place for the bitwise classes), the decision changes that would alter the claims, a timing-precision target (5% relative half-width for the verify-class kernels; the sub-20-microsecond commit kernels are reported but not gated), and the resampling unit (the frozen prefix); the 32-prefix confirmation captures were then taken under that frozen record.

The confirmation set (artifacts of record `experiments/out-20260922T051611Z-e7a-fresh-confirmation/`) contains 32 disjoint frozen prefixes, sixteen each with

TABLE VI

E7A KERNEL-LEVEL PILOT ACROSS NINE OPERAND SETS (FOUR HISTORICAL, FIVE SYNTHETIC); THE EIGHT FRESH, INDEPENDENTLY VERIFIED PREFIXES REPRODUCE THESE RANGES (TEXT). ERRORS ARE MAXIMA RELATIVE TO THE REFERENCE MAXIMUM, GIVEN AS THE RANGE OVER SETS; “FP32” AND “TF32” DENOTE THE PRECISION OF THE DENSE PRODUCTS IN B AND C. COMMIT ROWS COMPARE THE COMMITTED STATE FOR EVERY ACCEPTED PREFIX. BF16 AGREEMENT IS THE FRACTION OF DEPLOYED-WIDTH BFLOAT16 OUTPUT ELEMENTS BIT-IDENTICAL (INTEGER VIEW) TO THE NATIVE SPECULATIVE-UPDATE KERNEL’S. LOCAL REIMPLEMENTATIONS ONLY; NOT THE AUTHORS’ SYSTEMS.

Realization	Verify out	Commit (own)	Commit (oracle factors)	bf16 agreement
A scan + replay (production)	0.9–3.1e-7	1.0–3.7e-7	—	0.99998–1
B forward subst. fp32	2.8–5.2e-7	1.0–2.8e-7	1.0–2.1e-7	0.9997–0.9999
C finite Neumann fp32	2.8–5.2e-7	1.0–2.8e-7	1.0–2.1e-7	0.9997–0.9999
B forward subst. tf32	6.9e-4–1.4e-3	3.0–8.1e-4	2.6–8.9e-4	0.744–0.786
C finite Neumann tf32	6.5e-4–1.4e-3	3.0–8.1e-4	2.6–8.9e-4	0.745–0.788
June-2026 WY kernel (fp32 closed form)	1.4–2.1e-7	0.9–2.2e-7 (materialized)	—	—
Native speculative update (reference)	= A	1.0–3.7e-7	—	1
Native one-token decode vs. oracle	—	2.2e-4–1.7e-3	—	0.751–0.937
Native one-token decode vs. its own seam oracle	—	1.2–4.0e-7	—	—

256 and 4096 prompt tokens. One boot hung after autotuning and was retried after the driver’s timeout. One captured prefix, p087, failed the frozen provenance inspector: duplicate-token siblings made its path walk choose a different node from the served committer. That failure remains in the 32-prefix denominator. A revised path-consistency inspector passes the capture retrospectively; it does not replace the original verdict or add p087 to the 31-set primary comparison.

On the 31 provenance-passing sets, the production scan’s fp32-store output is bit-identical to the native speculative-update kernel’s, and the served bfloat16 output is byte-identical to the production scan’s. The maximum absolute compact-commit state errors are approximately 7.81×10^{-7} against the oracle and 1.91×10^{-6} against native; replay maxima are 1.34×10^{-6} and 2.38×10^{-7} , respectively (rounded). B and C’s bfloat16 outputs agree with native on at least 99.9772% of elements (the minimum is C on p005, with 14 of 61,440 elements differing). The served output is within 4.9×10^{-4} of the oracle and within one bfloat16 unit in the last place on significant elements. Sibling reordering and repeat replay remain bitwise invariant in their tested controls.

The frozen criteria are not satisfied. Native and production bfloat16 stores differ in one element on each of two sets, despite identical fp32 stores. One set exceeds the frozen absolute output tolerance by 3% (9.2×10^{-8} versus 8.9×10^{-8}). The frozen ordering $A \leq B = C$ for output error fails on three sets, and B/C compact-state error equality fails on p031 by 6.9×10^{-8} . Padding from 16 to 32 nodes changes output and correction-factor bits for B and C on all 31 sets. Timing also misses both declared checks: three blocks exceed 5% probe drift, and 14 verify-kernel timing cells across 12 prefixes exceed the 5% relative half-width limit, reaching 32.09%. Here “relative half-width” means $(p_{90} - p_{10}) / (2 \text{ median})$ across repeated kernel timings, a dispersion measure rather than a confidence interval.

A retrospective completeness audit found that the first checker omitted padding, compact-state ordering, and timing

half-width checks. The immutable freeze also incorrectly described the pilot as padding invariant and timing compliant: the referenced fresh-pilot-v2 data already fail padding on all eight prefixes and the half-width limit in seven timing cells across six prefixes (maximum 20.23%). We retain the original freeze and reports and record the corrections separately in `p0/monitor/2026-09-22-review-14-frozen-audit.json`. This is an audit of a flawed frozen analysis, not a corrected preregistration. These observations characterize the captured arithmetic; they do not qualify deployment, establish model-level harmlessness, or promote either compact route.

Cost is reported as kernel timing only, for one layer at B1, from the v3 run, whose 1 Hz compute-process inventory sampled no process other than our container and whose in-band probe drifted by at most 4.8% across timing blocks (2.7% in the tiny-gate run); rows perform different work and are not speedups. On the four historical sets (CUDA-event medians): the production scan takes 89 μs with its deployed bfloat16 output store and no state export; the B and C verifiers take 107–108 and 101–103 μs with matched bfloat16 output stores (109–111 and 101 μs with fp32 stores), both also writing their factors, and 41–42 and 31–32 μs with tf32 products; the replay commit of a depth-5 path writes six state rows in 102–105 μs , while the compact commit writes one row in 18–19 μs (54 μs for a zero-accept replay); the native speculative-update kernel run on the depth-5 chain with preallocated inputs takes 113–121 μs (its wrapper still allocates its outputs), and the native one-token decode kernel takes 24–25 μs per token. Across the tiny-gate sets, the sequential scan grows with node count (59 μs at six nodes, 90 at ten, 134 at twelve) while the B and C verifiers stay at 106–107 and 95–99 μs because their padded system is fixed. The recorded byte counts describe different quantities: the compact route stores 396 KB of factors per request and layer, while compact commitment writes one 3.1 MB state row and replay writes $(d+1)$ such rows. The tested legacy export allocates about 50 MB for a padded 16-node state tensor. These

storage and output-write counts do not measure peak live allocation or total memory traffic. Fresh-set kernel timings (shared-device diagnostic; owned container only sampled): production scan 88–90 μ s, B 105–107 and C 101–103 μ s with bfloat16 stores. Neither pilot shows model-level propagation, sampler acceptance, or continuation (E2/E7b), any serving effect (E1), or anything about the authors’ implementations.

G. Stage-isolated model diagnostics

We next isolated verifier substitution on one frozen pilot prefix, p072, at B1 with eager execution and the same pinned FP8 stack. Three separate boots ran the instrumented production route, a matched sham that computed B’s fp32 forward-substitution result without publishing it, and a verifier-only arm that published that result at layer 62. All three passed provenance checks. Across twelve contiguous, input-aligned steps with ten candidate rows each, the production and sham captures had identical logits, hidden states, residuals, and emitted token sequences. The thirteenth capture was partial at the response boundary and was excluded from the aligned comparison. This pair’s agreement is a control observation, not a universal cross-boot noise floor.

Against that sham, substitution first changed layer 62 and then layer 63; layers 0–61 remained identical. The largest absolute final-logit differences were 0.15625 at step 3 and 0.125 at step 7. No argmax changed among the 120 matched candidate rows, and all 32 emitted tokens were unchanged. Across thirteen verifier invocations, the minimum local bfloat16-output equality fraction was 99.9788%, with a maximum absolute difference of 6.10×10^{-5} . These observations demonstrate propagation in this implementation and prefix, without establishing model-level harmlessness or a defect inherent to either published mechanism. Raw captures and the independently checked reduction are retained under `experiments/out-20260922T063019Z-e7b-r13-p072-batch/`.

An independent review also found a publication defect in the diagnostic commit hook: on paths with more than one accepted draft, replacing only the accepted-column copy left the authoritative running state unchanged when the next forward read column zero. That defect concerns our substitution harness, not the compact algebra. After repair, three new boots ran production, a matched commit sham, and B’s commit-only substitution on p072 (`experiments/out-20260922T071945Z-e7b-r16-p072-commit-batch/`). Production and sham captures were bit-identical, including their 32 emitted tokens. All thirteen compact publications changed the authoritative state; all twelve captured next-forward pre-states consumed the preceding published bytes exactly. Across twelve input-aligned verifier calls, the compact arm changed final logits in two calls (maximum absolute difference 0.15625), without changing an argmax or the emitted token sequence. Only eleven calls had complete post-commit boundaries within the API output; the terminal partial call and extra capture are not additional fully emitted continuation intervals.

For a separate fixed-input check, we injected the compact arm’s first published layer-62 state into an offline float64

recurrence using the sham’s subsequent recorded operands and accepted paths, with fp32 state storage between steps. The initial maximum state difference, 2.38×10^{-7} , remained at that value over ten fully API-bound continuation intervals; maximum output differences ranged from 1.79×10^{-9} to 4.85×10^{-9} . The production-to-sham control stayed exactly zero. This once-injected, one-layer experiment differs from the live arm’s repeated compact commits and does not exercise cross-layer feedback. No amplification was observed in its tested maximum-state-error measure and horizon. All thirteen captured steps of the production and compact boots also met the declared layer-level numerical fidelity checks, and all twelve pre-state links were bit-identical to the prior publication. None of these checks qualifies convolution history, attention KV, the sampler law, or the full serving route; those surfaces remain separate E2 requirements. The recorded environments for all six diagnostic boots omit the plain-route attention-KV remap flag, whose loaded source defaults to disabled. Their local arithmetic and matched-arm observations therefore do not establish correct branch-path attention continuation; qualification must first repair and check that route.

VII. LIMITATIONS AND PLANNED VALIDATION

The archived and fresh measurements cover specific implementation stages, one main checkpoint family, and a limited task subset. The state contract is more general than its test coverage. Native-reference numerical agreement, sampler-law correctness, and deployment quality must remain separate claims. Instrumentation also has limits: H3 combines global structural-token counts with a pure-decode wall subset, so its proxy cannot establish matched decode or full-service throughput.

Table VII lists the selected validation scope and deferred extensions. After configuration reconciliation (P0), three core experiments compare arithmetic on identical inputs (E7a), test numerical propagation and accepted-prefix continuation (E2/E7b), and measure the frozen implementation against strong chains (E1). E3 is conditional on promoting unfinished optimizations. E7a has completed kernel measurements, including fresh pilot and confirmation operands. E2/E7b validation and E1 timing remain incomplete; diagnostic model boots do not qualify either experiment.

E7a compares local reimplementations of published mechanisms, not the authors’ systems. Identical captured operands and chosen paths isolate solver error from commit error. High-precision recurrence and each pinned native GPU path serve distinct reference roles. Its kernel-level pilot, the verified eight-prefix fresh-operand pilot, and the 32-prefix confirmation set evaluated against the record frozen from that pilot are reported in Section VI-F, including all identified frozen-rule failures and the retrospective audit of omitted checks. Those sample counts belong to E7a. Neither compact candidate met the frozen local criteria, so neither is promoted to an integrated serving or performance claim. E7b instead reports the bounded, single-prefix verifier-only and commit-only diagnostics and the offline fixed-input continuation in Section VI-G. The

TABLE VII

FINAL RECOMMENDED SCOPE. CORE EXPERIMENTS ARE E7A, E2/E7B, AND E1, PRECEDED BY P0; E3 IS CONDITIONAL. THE REMAINING ENTRIES ARE DEFERRED. E7A IS EXECUTED: KERNEL-LEVEL PILOT, VERIFIED EIGHT-PREFIX FRESH-OPERAND PILOT, AND 32-PREFIX CONFIRMATION AGAINST A FROZEN RECORD (SECTION VI-F); THE REMAINING CORE RESULTS ARE UNQUALIFIED OR PENDING, AND DIAGNOSTIC MODEL BOOTS DO NOT ESTABLISH QUALIFICATION.

ID	Question	Minimum comparison	Claim enabled
P0	What actually ran?	Archived binary, flags, sampling and denominator reconciliation	Correct configuration labels
E1	Does the final tree beat strong chains?	Cat10, depth-comparable chain-5, longer chain-11; actual B1/B4; paired independent boots	Current-stack performance
E7a	Do arithmetic realizations agree?	Same operands: sequential, triangular solve, finite-Neumann; output and commit isolated	Numerical and local-cost characterization
E2/E7b	Do errors amplify or corrupt continuation?	Fixed-prefix stage substitutions; layer/final logits; state/KV/conv, sampler and cache fixtures	Scoped continuation and implementation agreement
E3	Do optimizations combine?	Conditional scan/accept-walk toggles, individually and together	Composed local speedup
E4	How does memory affect service latency?	Deferred: identical arrival trace and cache-capacity controls	Serving and cache explanation
E5	Is broader task quality preserved?	Deferred: held-out tasks and powered quality design; exact sampler checks stay in E2	Broader quality evidence
E6	Does the result extend to NVFP4?	Deferred: matched native control and repeated composed tests	Quantization extension
E7c	How do authors' systems compare?	Deferred: feasible matched external implementations	Competitive system comparison

matched input-aligned windows are observed controls, not a general forced-path facility: the served route refuses the older forced-spine diagnostic. No combined-candidate confirmation campaign or powered task-quality study has been completed. Broader promotion would require those separate verification, commitment, combined-route, and continuation checks; the present failures remain reportable numerical findings.

E1 uses controlled prefix replay rather than conflating it with fixed-arrival serving or live agent execution. Actual emitted tokens and retained wall intervals must be bound to the same event identifiers; component spans must match those events before attributing their residual to host work. Three arms, two actual batch sizes, and three independent paired boots give an initial 18 timing cells. If a reconstruction route passes E2/E7b and is integrated, retain the sequential tree as an ablation and add that route as a fourth arm, yielding 24 cells. Replication is set from pilot variance and a prespecified practical precision target; the final test is not repeated until a favorable result appears.

E2 should map existing coverage before adding tests. The repository already includes an analytic multidraft sampler identity, multistep sampling tests, a biased-selector negative control, and accepted-path state fixtures [10]. Missing coverage concerns their connection to the current deployed GPU route, including zero/full acceptance, sibling paths, page reuse, and pending-token continuation. Source availability alone is not a passing execution record. No proof of full distribution preservation is inferred from the historical numerical or garble checks.

VIII. CONCLUSION

GDN Tree-Scan studies the implementation boundary between tree verification and the next native serving iteration. A correct continuation requires agreement among recurrent state, convolution history, attention KV, physical row mappings, and the sampler. The archived failures show that satisfying one surface can leave another inconsistent.

The KV-remap fixture improved from 15/15 failures to 0/15, and the numerical investigation produced a shared-spine repair with byte agreement in its tested checks (Table III). In the later three-arm campaign, the tree's archived structural-token/pure-wall proxy was 32.85 versus native MTP-5's 42.74 (Table IV). The underlying event populations differ, so those values preserve an accounting observation rather than establish a measured throughput ranking. A separate paired attention optimization reduced its tested B4 step by 27.03 ms (Table V) [10].

These findings support a scoped account of state integrity, numerical execution, and serving cost. They do not establish superiority over direct GDN tree methods or a distribution-preservation guarantee for the current implementation. Configuration reconciliation has documented unresolved historical binary identity and duplicated sampling constraints. Fresh same-input arithmetic adds bounded numerical evidence, while matched current-stack timing and composed correctness checks remain incomplete.

REFERENCES

- [1] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," 2022. [Online]. Available: <https://arxiv.org/abs/2211.17192>
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating Large Language Model Decoding with Speculative Sampling," 2023. [Online]. Available: <https://arxiv.org/abs/2302.01318>
- [3] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, "Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification," 2024. [Online]. Available: <https://arxiv.org/abs/2305.09781>
- [4] S. Yang, J. Kautz, and A. Hatamizadeh, "Gated delta networks: Improving mamba2 with delta rule," 2025. [Online]. Available: <https://arxiv.org/abs/2412.06464>
- [5] Y. Wu, Z. Qin, A. Wong, and S. Soatto, "Stree: Speculative tree decoding for hybrid state-space models," 2025. [Online]. Available: <https://arxiv.org/abs/2505.14969>
- [6] Z. Ma, "GDN Tree-Scan consolidated implementation and measurement receipts," https://github.com/MaCoredroid/Lumo_FlyWheel/tree/55f55854328b37f262e97d57b5863d8fadd7ff76, 2026, commit-pinned repository receipts for deploy TPS, equivalence gates, runtime, launchers, kernels, and configuration.

- [7] —, “Stateless Tree Speculative Decoding: A Branched Verifier That Caches and Commits Like Native,” https://macoredroid.github.io/Lumo_FlyWheel/stateless-tree.html, 2026, author experiment archive; accessed 2026-09-21.
- [8] —, “One Bit in the Last Place: Why a Wider Speculative Tree Accepted Fewer Tokens,” https://macoredroid.github.io/Lumo_FlyWheel/spine-reorder.html, 2026, author experiment archive; accessed 2026-09-21.
- [9] —, “Keep or Replay: Why a Speculative Tree That Accepts More Still Loses to the Linear Chain,” https://macoredroid.github.io/Lumo_FlyWheel/keep-or-replay.html, 2026, author experiment archive; accessed 2026-09-21.
- [10] —, “Lumo FlyWheel implementation and experiment archive,” https://github.com/MaCoredroid/Lumo_FlyWheel/tree/984f613d7885d9bf7898866827552b68a3199d0c, 2026, revision evidence snapshot; individual campaigns retain their own run identities.
- [11] —, “Twenty-Seven Milliseconds, Shipped,” https://macoredroid.github.io/Lumo_FlyWheel/numbers-that-didnt-survive.html, 2026, author experiment archive; accessed 2026-09-21.
- [12] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen, “Sequoia: Scalable, robust, and hardware-aware speculative decoding,” 2025. [Online]. Available: <https://arxiv.org/abs/2402.12374>
- [13] Y. Li, F. Wei, C. Zhang, and H. Zhang, “Eagle-2: Faster inference of language models with dynamic draft trees,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.16858>
- [14] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan, Z. Wang, and M. Zhang, “Opt-tree: Speculative decoding with adaptive draft tree structure,” 2025. [Online]. Available: <https://arxiv.org/abs/2406.17276>
- [15] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [16] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient Execution of Structured Language Model Programs,” 2023. [Online]. Available: <https://arxiv.org/abs/2312.07104>
- [17] SGLang Team, “Hybrid Models Meet SGLang: More than Full Attention,” <https://pytorch.org/blog/hybrid-models-meet-sglang-more-than-full-attention/>, 2025.
- [18] SGLang Project, “Qwen3-Next SGLang documentation,” <https://lmsys.org/mintlify.app/cookbook/autoregressive/Qwen/Qwen3-Next>, 2026.
- [19] T. Dao, “Flashattention-2: Faster attention with better parallelism and work partitioning,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.08691>
- [20] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim, “Parallelizing Linear Transformers with the Delta Rule over Sequence Length,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.06484>
- [21] L. Zhong, S. Xu, H. Wen, T. Xie, Q. Guo, Y. Wang, and M. Li, “Specmamba: Accelerating mamba inference on fpga with speculative decoding,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.19873>
- [22] H. Borobia, E. Seguí-Mas, and G. Tormo-Carbó, “Component-aware self-speculative decoding in hybrid language models,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.01106>
- [23] L. Wang, Y. Su, X. Wu, C. You, Y. Liu, Z. Qiu, J. Zhang, J. Zheng, F. Liu, J. Zhang, C. Tian, and C. Huan, “Bole: Efficient Tree Speculation for Hybrid-Attention Language Models,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.01651>
- [24] S. M. Ghantasala, “TreeWY: Speculative Verification for Gated DeltaNet Hybrids,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.20961>
- [25] Dao AI Lab and NVIDIA, “ReplaySSM: Cache SSM Inputs Instead of State,” <https://github.com/vllm-project/vllm/issues/47572>, 2026, implementation RFC; accessed 2026-09-21.
- [26] Qwen Team, “Qwen3.6-27B-FP8,” <https://huggingface.co/Qwen/Qwen3.6-27B-FP8>, 2026, public Hugging Face model page; accessed 2026-09-21.
- [27] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” 2023. [Online]. Available: <https://arxiv.org/abs/2310.06770>
- [28] Z. Ma, “Every Lever We Pulled: A Speculative-Decoding Speed Campaign in Full,” https://macoredroid.github.io/Lumo_FlyWheel/everylever.html, 2026, author experiment archive; accessed 2026-09-21.